

Chapter 4

Web Services and APIs

1. API basics: Role in web applications
2. REST principles and design, RESTful APIs
3. JSON versus XML data exchange
4. Data validation and serialization
5. Microservices

Duration: 7 Hours

Marks Distribution: 9 Marks

API Basics: Role in Web Applications

An API (Application Programming Interface) is essentially an interface for code, written with more code. A request can be made to an API using (for example) a URL. This will trigger a certain piece of code and this piece of code will give a certain response back.

APIs work with Hypertext Transfer Protocol (HTTP) calls. HTTP is just a protocol for communication between two parties: a client and a server, or a server and another server (in which the requesting server acts like the client). This means that it has to stick to certain conventions and rules that the other party expects, and the other party will respond in a certain way. For example, this means using a specific format to specify headers, using GET methods for getting information, using POST methods for creating new information on the server, and using PUT methods to change information on the server.

API is a set of rules, protocols, and tools that allows one software application to communicate with another. Acts as a **bridge** between different software systems.

Roles of APIs in Web Applications

1. **Data Access:** APIs allow applications to fetch, create, update, or delete data from databases or other services.
2. **Integration:** Enable apps to integrate with third-party services (e.g., payment gateways, social media).
3. **Abstraction:** Hide the internal workings of an application; clients only interact with the exposed endpoints.
4. **Automation:** APIs allow tasks to be automated, e.g., updating data from multiple sources.
5. **Scalability & Flexibility:** Apps can grow independently and communicate through APIs without tight coupling.

REST Principles and Design, RESTful APIs

- REST (Representational State Transfer) is an architectural style for designing networked applications.
- It was introduced by Roy Fielding in his doctoral dissertation in 2000.
- REST is not a protocol; it's a set of constraints that guide how web services should behave to be scalable, simple, and efficient.
- REST is based on the concept of **resources**.
- Everything that the client wants to interact with is considered a **resource** (e.g., users, products, orders).
- Each resource is uniquely identified by a **URI (Uniform Resource Identifier)**.
- Eg:

Resource: User with ID 101

URI: <https://api.example.com/users/101>

Key Principles of REST

1. **Stateless:** Each request from client to server must contain all information; server does not store session info. Each request from the client must contain **all necessary information**. This makes REST scalable.
2. **Client-Server Architecture:** Separation of client (UI) and server (data storage & logic). Client and server are separate. The client handles UI, server handles data and logic. This separation allows **scalability and flexibility**.
3. **Cacheable:** Responses must define whether they can be cached to improve performance.
4. **Uniform Interface:** Standard methods and formats for resources. Standardized way for clients and servers to communicate. Simplifies development. It includes:
 - a. Resource identification (URI)
 - b. Manipulation through representations (JSON/XML)
 - c. Self-descriptive messages
5. **Layered System:** Server may be part of a hierarchy of servers, improving scalability and security. Clients cannot tell if it communicates with the end server

or intermediary servers (like load balancers). Supports scalability and security.

- 6. Code on Demand (optional):** Servers can send executable code to clients if needed (e.g., JavaScript snippets).

Restful API

A RESTful API is an API that follows REST principles. It allows clients to create, read, update, and delete (CRUD) resources over HTTP.

Key Components of Restful APIs

1. HTTP Methods Used in REST

Method	Purpose	Example
GET	Retrieve resource(s)	GET /users/101
POST	Create new resource	POST /users
PUT	Update a resource completely	PUT /users/101
PATCH	Update a resource partially	PATCH /users/101
DELETE	Delete a resource	DELETE /users/101

2. Resource Representation

- Resources are identified via **URIs (Uniform Resource Identifiers)**.
- Common formats: **JSON, XML**.

```
{  
  
    "id": 101,  
  
    "name": "Sunil Banmala",  
  
    "email": "sunil@example.com"  
  
}
```

- Example URI: /users/123 → Access user with ID 123.

Example of a RESTful API Workflow

Imagine an **e-commerce app**:

1. Get all products

GET /products

Response:

```
[  
  
    {"id": 1, "name": "Tea", "price": 50},  
  
    {"id": 2, "name": "Coffee", "price": 100}  
  
]
```

2. **Get a single product**

GET /products/1

Response:

```
{"id": 1, "name": "Tea", "price": 50}
```

3. **Add a new product**

POST /products

Body:

```
{"name": "Green Tea", "price": 70}
```

Response:

```
{"id": 3, "name": "Green Tea", "price": 70}
```

4. **Update a product**

PUT /products/3

Body:

```
{"name": "Herbal Green Tea", "price": 80}
```

Response:

```
{"id": 3, "name": "Herbal Green Tea", "price": 80}
```

5. Delete a product

DELETE /products/3

Response: 204 No Content

Advantages of RESTful APIs

1. **Scalability:** Stateless nature allows horizontal scaling.
2. **Flexibility:** Can support multiple data formats (JSON, XML).
3. **Performance:** Lightweight and cacheable.
4. **Ease of Integration:** Works well with web, mobile, and IoT applications.
5. **Maintainability:** Uniform interface simplifies updates and testing.

Best Practices for REST APIs

- Use **plural nouns** for resource names: /users not /user
- Use proper **HTTP status codes**:
 - 200 → OK
 - 201 → Created
 - 400 → Bad Request
 - 404 → Not Found
 - 500 → Internal Server Error
- Version your API: /v1/users
- Secure APIs with authentication (OAuth, JWT)
- Provide **meaningful error messages** for debugging
- Using cookies to store tokens rather than in local storage and sessions

JSON versus XML Data Exchange

Feature	JSON (JavaScript Object Notation)	XML (Extensible Markup Language)
Readability	Easy for humans to read	Verbose, harder to read
Data Size	Lightweight, smaller payloads	Larger, more overhead
Parsing	Faster, supported by JavaScript natively	Slower, requires parsers
Format	Key-value pairs, arrays, objects	Markup tags, hierarchical tree
Usage	Widely used in modern APIs	Older systems, SOAP APIs

Summary: JSON is preferred today for RESTful APIs due to its simplicity, speed, and native support in JavaScript.

```
{
```

```
  Id: 3345
```

```
  name:"sunil"
```

```
}
```


Data Validation and Serialization

Data Validation

- Ensures that incoming/outgoing data is **correct, complete, and safe**.
- Common checks:
 - Required fields present
 - Correct data type (integer, string, date)
 - Constraints (min/max values, length)
 - Format validation (email, phone, URL)

Serialization

- Process of **converting data structures into a format suitable for transmission** (e.g., JSON or XML).
- **Deserialization**: Converting the received data back into a usable object in the program.

Microservices

Microservices is an architectural style where an application is built as a **collection of small, independent services**, each responsible for a specific feature.

Characteristics

1. **Single Responsibility:** Each service focuses on one business capability.
2. **Independently Deployable:** Services can be developed, deployed, and scaled separately.
3. **Decentralized Data Management:** Each service may manage its own database.
4. **Communication via APIs:** Services interact over HTTP/REST or message queues.
5. **Scalability:** Individual services can scale as needed.

Advantages

- Easier to maintain and update
- Supports multiple technologies (polyglot programming)
- Fault isolation: One service failure doesn't crash the entire app

Challenges

- Complexity in managing multiple services
- Requires robust **service discovery** and **API management**
- Distributed system issues: latency, security, and transactions

Summary Table

Topic	Key Points
API Basics	Bridge between apps, enables integration, data access, automation
REST Principles	Stateless, client-server, cacheable, uniform interface, layered system
RESTful API Methods	GET, POST, PUT, PATCH, DELETE
JSON vs XML	JSON is lightweight, faster, readable; XML is verbose, older systems
Data Validation & Serialization	Ensure correctness, convert to/from JSON/XML for transmission
Microservices	Independent, small services; scalable, maintainable, communicate via APIs